LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 3 – Konsep Dasar Pemrograman Berorientasi Objek

NAMA: Anindya Sukma Dwiyanda

NIM: 103132400006

KELAS: S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman Berorientasi Objek (OOP) adalah paradigma pemrograman yang membungkus data dan fungsi ke dalam komponen objek dengan prinsip utama seperti abstraksi, enkapulasi, pewarisan, dan polimorfisme. Dalam Java, class bertindak sebagai cetak biru yang mendefinisikan class field (variable bersama milik kelas) dan instance field (variable independent milik objek). Objek berinteraksi melalui method dengan keyword this untuk merujuk pada instance yang sedang aktif. Selain itu, mekanisme pengiriman argumenberlaku secara pass by value untuk tipe data primitif dan pass by reference untnuk objek atau array, sehingga perubahan data array di dalam method secara langsung memperbarui data asli pada memori.

B. Guided

1. Guided 1 - Abstraksi

```
package guided;

public class Abstraksi {
    public static void main(String[] args) {
        Kucing oren = new Kucing();
        oren.nama = "Cimot Gendot";
        oren.ras = "Angora";
        oren.bersuara();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Laprak_Week_3 ---
Cimot Gendot Angora Meow
```

2. Guided 2 - Buku

```
package guided;

public class Buku {
    private String pengarang;
    private String judul;
    private Buku() {
        this("Rumah Kita", "GoodBles");
          // this ini digunakan untuk memanggil konstruktor yang
          // menerima dua parameter
    }
    private Buku(String judul,String pengarang)
    {
        this.judul = judul;
        this.pengarang = pengarang;
    }
    private void cetakKeLayar()
```

```
    {
        System.out.println("Judul : " + judul + " Pengarang : " + pengarang);
    }
    public static void main(String[] args)
    {
        Buku a,b ;
        a = new Buku("Jurassic Park", "Michael Chricton");
        b = new Buku();
        a.cetakKeLayar();
        b.cetakKeLayar();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Laprak_Week_3 ---
Judul : Jurassic Park Pengarang : Michael Chricton
Judul : Rumah Kita Pengarang : GoodBles
```

3. Guided 3 – Circle01

```
package guided;

public class Circle01 {
    public static final double PI = 3.14;

    public static double radiansToDegrees(double rads) {
        return rads * (180 / PI);
    }

    public double r;

    public double area() {
        return PI * r * r;
    }

    public double circumference() {
        return 2 * PI * r;
    }

}
```

4. Guided 4 – Demo Lagu

```
package guided;

public class DemoLagu {
    public static void main(String[] args) {
        Lagu a = new Lagu();
```

```
        a.IsiParam("God Will Make A Way", "Don Moen ");
        a.cetakKeLayar();
    }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ Laprak_Week_3 ---
Judul : God Will Make A Way, pencipta : Don Moen
```

5. Guided 5 – Kucing

```
package guided;

public class Kucing {
    String nama;
    String ras;

    void bersuara() {
        System.out.println(nama + " " + ras + " Meow");
    }

}
```

6. Guided 6 – Enkapulasi

```
package guided;

public class Enkapsulasi {

    public static void main(String[] args) {
        Rekening rek = new Rekening();
        rek.tambahsaldo(300000000);
        rek.tampilkansaldo();
    }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ Laprak_Week_3 ---
Saldo berhasil ditambahkan: 300000000
Saldo saat ini: 300000000
```

7. Guided 7 – Pass By Value

```
package guided;

public class PassedByValue {
   public static void main(String[] args) {
  int x,y;
        TestPass z;
        z = new TestPass(50,100);
        x = 10;
        y = 20;

        System.out.println("Nilai sebelum passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        //passed by value
        z.calculate(x,y);
        System.out.println("Nilai sesudah passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);
        System.out.println("Nilai sebelum passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);

        //passed by reference
        z.calculate(z);

        System.out.println("Nilai sesudah passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);
   }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ Laprak_Week_3 ---
Nilai sebelum passed by value :
x = 10
y = 20
Nilai sesudah passed by value :
x = 10
y = 20
Nilai sebelum passed by reference :
z.i = 50
z.j = 100
Nilai sesudah passed by reference :
z.i = 500
z.j = 50
```

C. Unguided

1. Soal Unguided 1

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di ‘main’ sebelum diserahkan untuk diolah. Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun bukan sesuatu yang berdeda-beda tiap kali data dibuat sehingga sebaiknya ditulis satu kali sjaa sebagai aturan tetap milik class, bukan diketik ulang sebagai angka ‘-1.0’ di setiap method yang membutuhkannya. Buatlah satu class yang bertugas mengolah data suhu ini memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistic dasar dari data yang sudah bersih.

Jawaban :

```
package unguided3;

import java.util.Arrays;

public class PengolahSuhu {

    // Instance field (hanya dapat diakses melalui method dalam class ini)
    private double[] suhuHarian;

    // Class field / konstanta (bernilai sama untuk semua objek dan tidak dapat
diubah)
    public static final double NILAI_KOSONG = -1.0;

    // Constructor dengan nama parameter yang sama dengan nama instance field
    public PengolahSuhu(double[] suhuHarian) {
        // Penggunaan keyword 'this' secara eksplisit untuk membedakan field dan
parameter
```

```
        // Constructor ini menyimpan referensi alamat memori dari array, bukan
membuat salinan baru
        this.suhuHarian = suhuHarian;
    }

    // Method untuk menampilkan seluruh data suhu 7 hari
    public void tampilkanData() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                System.out.println("Hari " + (i + 1) + " : (kosong)");
            } else {
                System.out.println("Hari " + (i + 1) + " : " + suhuHarian[i] + "°C");
            }
        }
    }

    // Method untuk mencari index data yang bernilai NILAI_KOSONG (-1.0)
    public int cariIndexKosong() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                return i; // Mengembalikan index jika ditemukan
            }
        }
        return -1; // Mengembalikan -1 jika tidak ada data kosong
    }

    // Method untuk mengisi data kosong menggunakan rumus imputasi deret
waktu
    public void isiDataKosong() {
        int indexKosong = cariIndexKosong();

        // Memastikan index kosong ditemukan dan memiliki elemen sebelum serta
sesudahnya
        if (indexKosong != -1 && indexKosong > 0 && indexKosong <
suhuHarian.length - 1) {
            suhuHarian[indexKosong] = (suhuHarian[indexKosong - 1] +
suhuHarian[indexKosong + 1]) / 2.0;
        }
    }

    // Method untuk menghitung rata-rata suhu harian
    public double hitungRataRata() {
        double total = 0.0;
        for (double suhu : suhuHarian) {
            total += suhu;
        }
        return total / suhuHarian.length;
    }

    // Main method sebagai titik awal jalannya program
    public static void main(String[] args) {
```

```
        // Data suhu awal harian
        double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };

        // Membuat objek PengolahSuhu
        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        // Menampilkan data suhu awal
        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();
        System.out.println();

        // Mencari dan menampilkan index hari kosong
        int indexKosong = pengolah.cariIndexKosong();
        System.out.println("Index hari kosong (dimulai dari 0): " + indexKosong);
        System.out.println();

        // Mengisi data kosong dengan metode imputasi
        pengolah.isiDataKosong();

        // Menampilkan data suhu setelah proses pengisian
        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();
        System.out.println();

        // Menampilkan hasil perhitungan rata-rata suhu
        System.out.printf("Rata-rata : %.2f°C\n", pengolah.hitungRataRata());
        System.out.println();

        // Mencetak langsung variabel array suhuHarian milik method main
        System.out.println("Isi array suhuHarian di main setelah isiDataKosong()
    dijalankan:");
        System.out.println(Arrays.toString(suhuHarian));
        System.out.println("(ikut berubah: constructor menyimpan referensi array
    yang sama)");
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Laprak_Week_3 ---
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29,39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
```

Penjelasan :
Program PengolahSuhu.java mengimplementasikan prinsip enkapulasi dengan mengisolasi atribut array suhuHarian secara private dan menyediakan variable static final NILAI_KOSONG sebagai konstanta milik class. Penggunaan keyword this pada konstruktor secara eksplist membedakan atribut objek dari parameter nama sejenis. Program ini menjalankan proses pencarian data hilang bertanda -1.0 lalu melakukan imputasi raata-rata tetangga terdekat, serta memanfaatkan sifat pass by reference pada tipe data array sehingga modifikasi nilai di dalam objek otomatis memperbarui data array asal pada method.

D. Kesimpulan

Secara keseluruhan, Modul 3 mengajarkan dasar paradigma Pemrograman Berorientasi Objek (OOP) melalui penerapan class sebagai cetak biru dan object sebagai bentuk nyata dari cetak biru tersebut di memori. Modul ini menanamkan pentingnya prinsip enkapsulasi untuk melindungi data internal, membedakan penggunaan class field (static) dan *instance* field, serta memanfaatkan keyword this untuk merujuk pada instance yang sedang aktif. Selain itu, pemahaman mengenai pengiriman argumen pass by value untuk

tipe primitif dan pass by reference untuk tipe referensi (seperti array dan objek) menjadi kunci utama dalam memahami bagaimana data disimpan dan dimanipulasi di dalam memori komputer.